RUA - Routing UA

universidade de aveiro
theoria poiesis praxis

instituto de telecomunicações

Milestone 4: System Component Validation (Performance, Scalability, Testing, Security)

PEI 2025/2026

Group 7:

Tomás Gameiro - 119586

Francisco Silva - 118716

João Cabral - 119696

Gonçalo Floro - 120189

Bernardo Martins - 120151

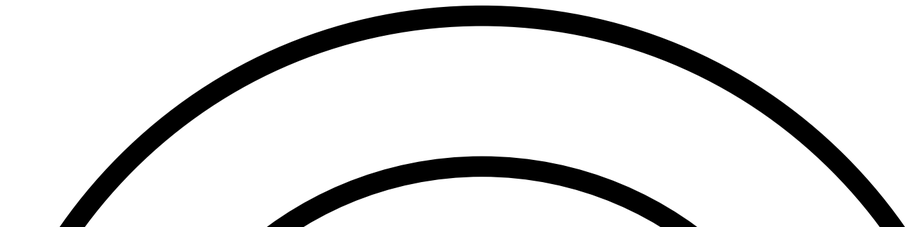
Supervisors:

Rui L. Aguiar

Daniel Corujo

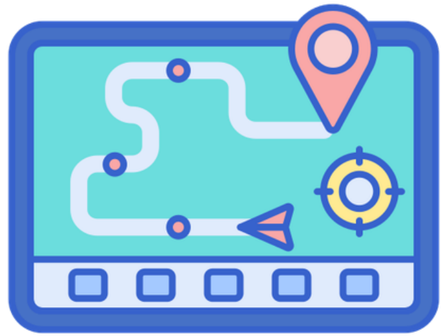
Abdelgader Mahmoud

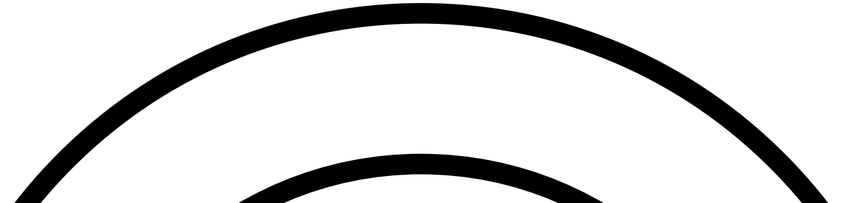
Overview

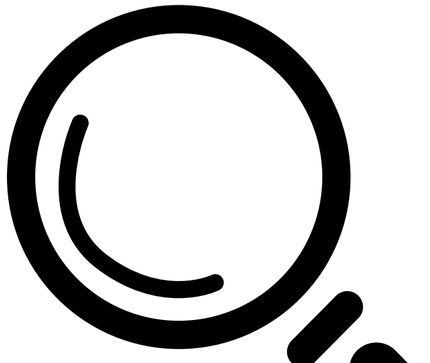


Context and Goals

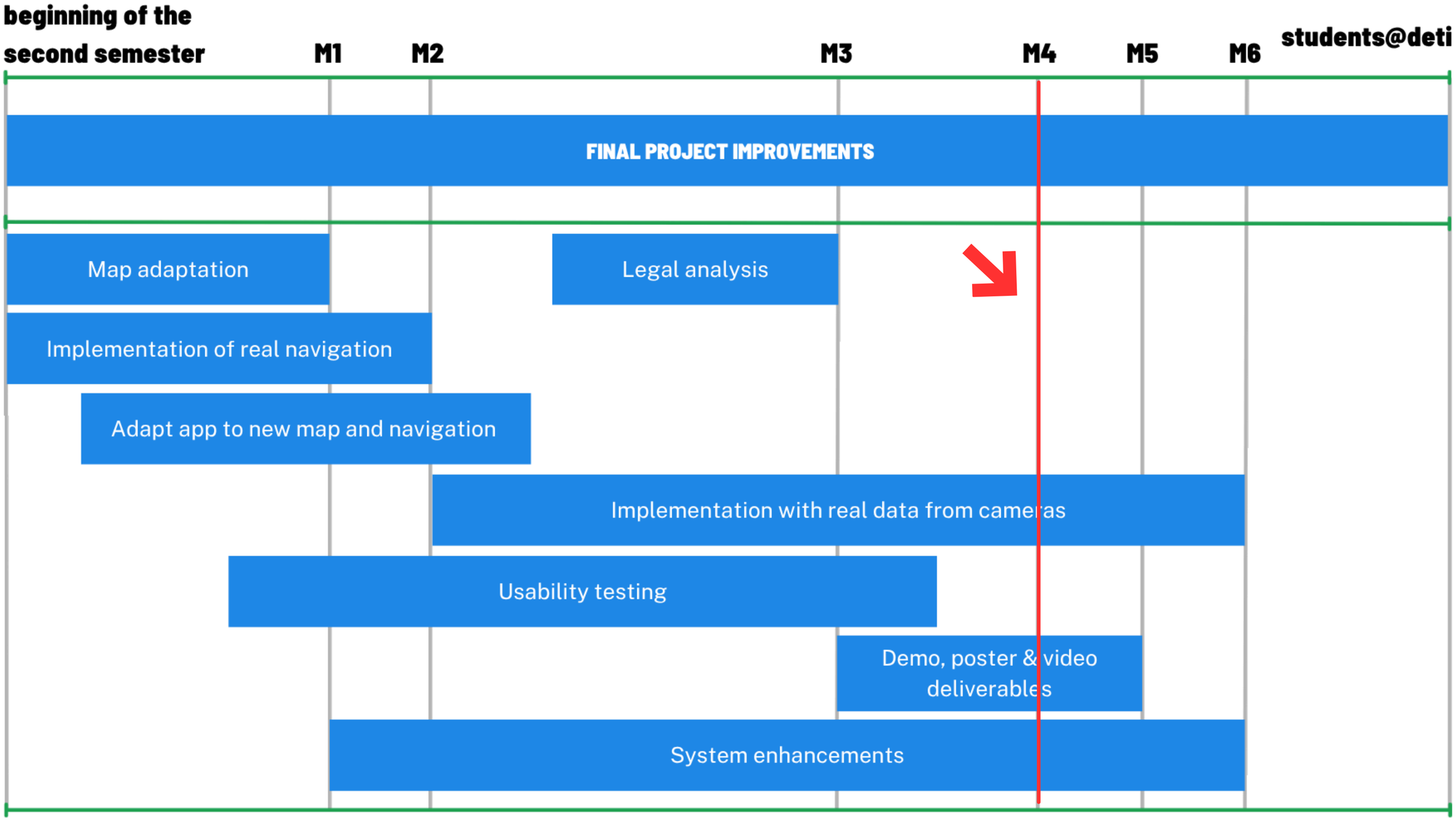
- Improve campus navigation for students, staff, and visitors
- Provide real-time outdoor navigation
- Ensure safety with accessible and emergency evacuation routes
- Provide real-time congestion's maps and queues of UA







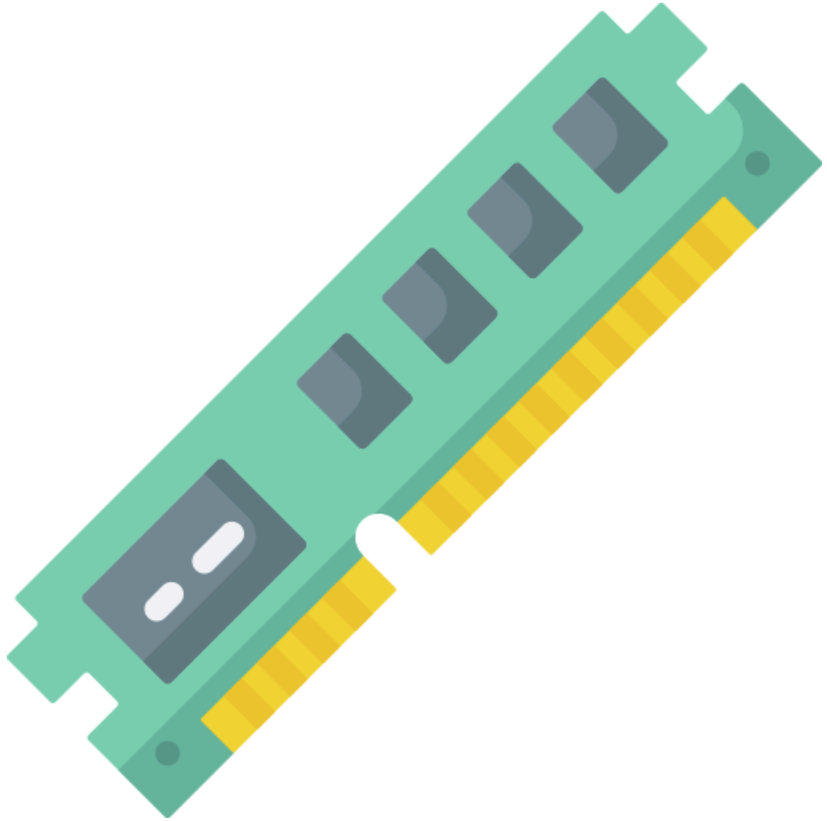
Calendar


beginning of the second semester
M1
M2
M3
M4
M5
M6
students@deti
FINAL PROJECT IMPROVEMENTS
Map adaptation
Legal analysis
Implementation of real navigation
Adapt app to new map and navigation
Implementation with real data from cameras
Usability testing
Demo, poster & video deliverables
System enhancements

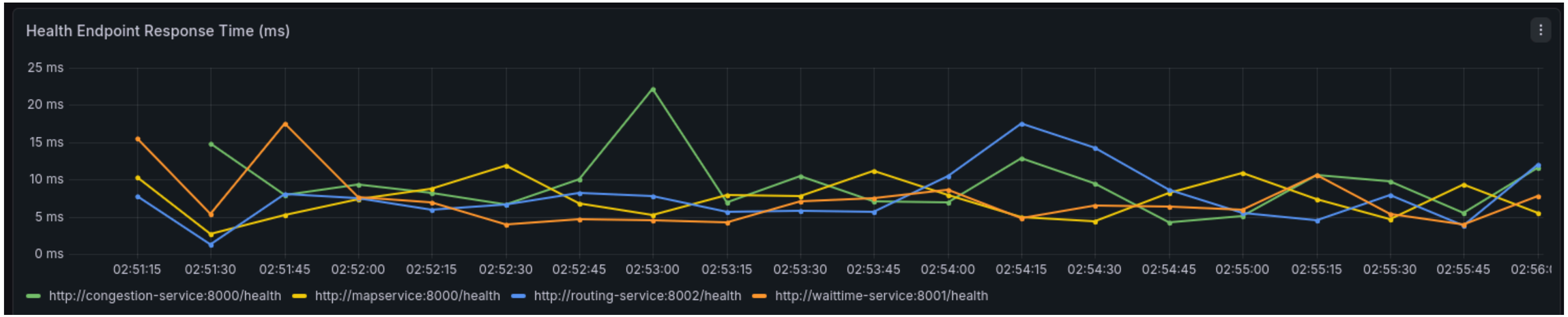
Performance

VM Specs

- 16 GB RAM
- 4 CPU cores
- 1TB HDD



Response Time Metrics



- 95% of requests complete in under 187ms, well within the 1s threshold
- 100% health checks passed: all 264 checks passed across all services

Throughput Metrics

- **Under normal conditions (50 users):** 30 requests/second, average response time of 653ms

- **Under stress (1000 users):** 174.81 requests/second, but with significant degradation - average response time of 3.69s

Slow Queries

- **Identified slow queries: GET /api/pois and GET /api/poi/{id}** - both fetch all nodes from the database and then search

- **Proposed optimization:** add database indexes and filter at query level - reducing full table scans and improving response time under load

Scalability

Stress Testing

- **Only Congestion and Routing:** resisted without critical errors

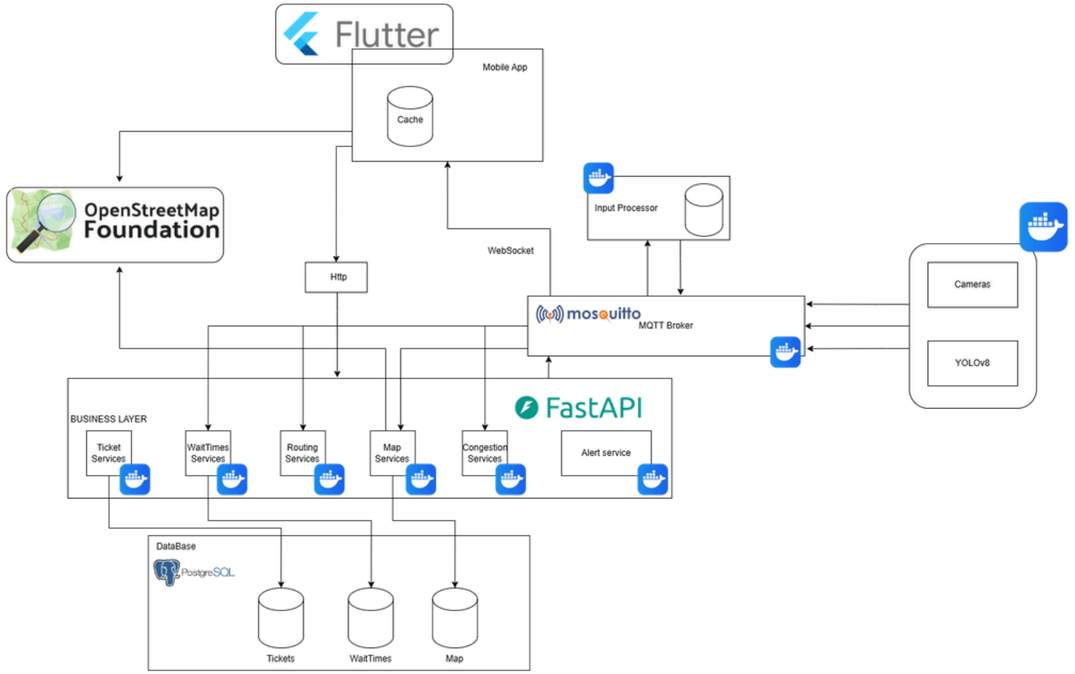
What's broken and why

- **Map Service:** collapsed under high load. Being called by multiple services simultaneously, its failure had a cascading effect across the entire system

- **Wait-Time Service:** degraded with 1000 simultaneous users

Architectural Proof

- The system is separated into independent services, each with its own isolated environment, allowing horizontal scaling of each service individually.



Security

Dependency Audit

- Scanned all services for known dependency vulnerabilities
- 10 vulnerabilities turned to 0, all fixed through dependency updates

OWASP Top 10

- **Injections:** SQLAlchemy ORM used throughout. Raw SQL queries are impossible by design

- **Input Validation:** Pydantic automatically rejects malformed inputs at API level. Invalid coordinates, wrong types and missing fields never reach business logic

OWASP Top 10

- **Broken Access Control:** No admin endpoints. All routes are public and anonymous by design, no privilege escalation possible

- **Security Misconfiguration:** Each microservice runs is isolated with MQTT ACLs enforced - only authorized services can publish and subscribe to their designated topics

MQTT Security

- **TLS implemented:** all MQTT communications (broker, services, and Flutter app) are now encrypted end-to-end.

- **Broker locked down:** anonymous access disabled; only the Flutter app and internal services can connect over encrypted channels.

Data Protection

- **Camera Privacy:** Cameras only publish people counts to the MQTT broker - no images are ever stored or transmitted

- **GPS Data:** User location data automatically expires after 20 seconds of inactivity


```
{ "event_id": "f5b8a9c3-1d2a-4f5b-9d8c-
7e6d5c4b3a2f",
  "event_type": "queue_update",
  "location_type": "WC",
  "location_id": "Q_WC_Norte",
  "queue_length": 5,
  "timestamp": "2026-05-17T22:01:35Z",
  "metadata": { "camera_id": "CAM_1" } }
```

AI Model Validation

Performance Metrics

- **Precision:** 0.54 | **Recall:** 0.26 | **F1-Score:** 0.355
- Real-time guaranteed: 31.4 ms average latency, >28 FPS sustained.
- All embedded resource limits respected (RAM 248 KB / Flash 11.7 MB).

Cross-Validation & Boundary Testing

- **5-Fold cross-validation:** minimal deviation between folds (no overfitting, good generalization)

- **Boundary testing passed:** completely black frames, white frames, and Gaussian noise (all handled stably).

Manual Inspection

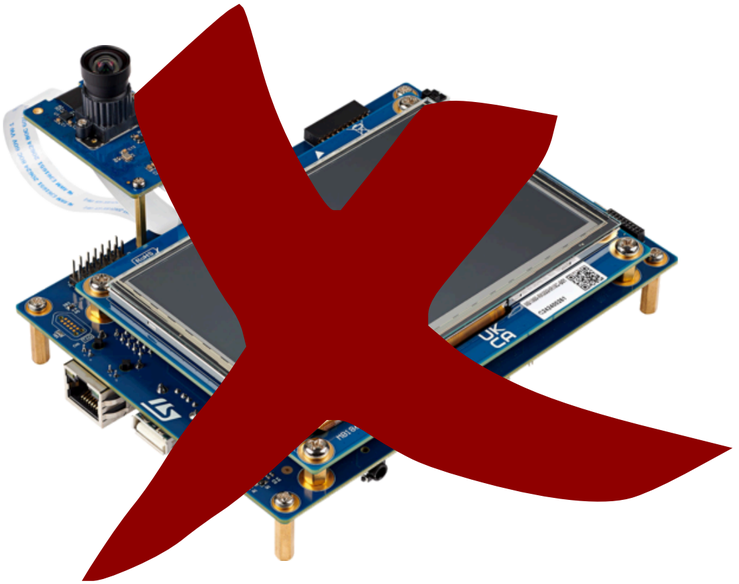
- **Crowds & tiny persons:** merged bodies visually fuse; persons under 4 px missed due to coarse 19×19 feature maps.

- **Occlusion & darkness:** bodies hidden lack visible features, so the visual spectrum loses critical contrast.

Software Project Management

Planning Issues in our project

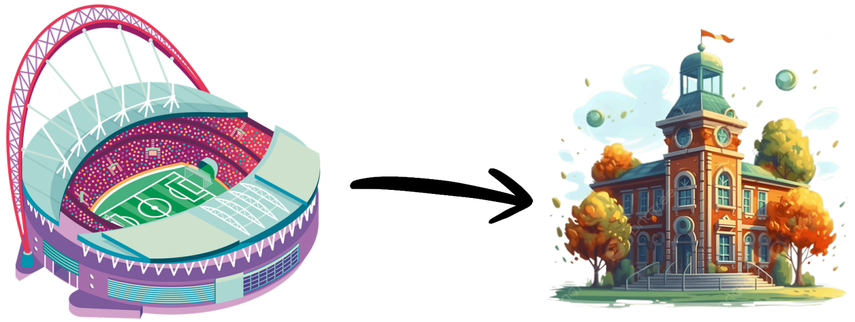
Throughout most of the sprints our Camera-hardware focused tasks kept delaying due to those not being available



Planning Issues in our project

Some of the tasks also spilled to later sprints due to underestimations of the time they would take, like:

- Tasks to do with the adaptation of Map/Routing Service to campus context
- Frontend features



Planning Issues in our project

- Our planning had to be very versatile due to all the changes that occured in our project throughout the two semesters: Stadium → IT Indoor → UA Campus

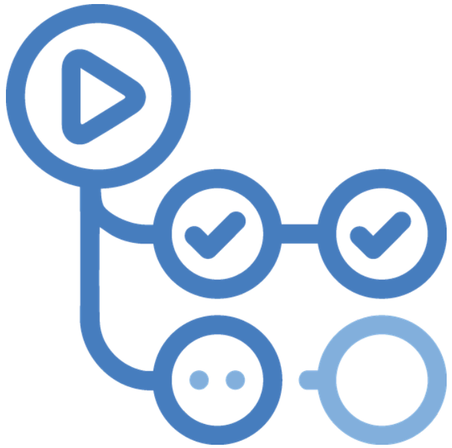
- Everytime a sprint ended or a task needed changes, meetings were scheduled to reorganize the work in group

Automation in CI

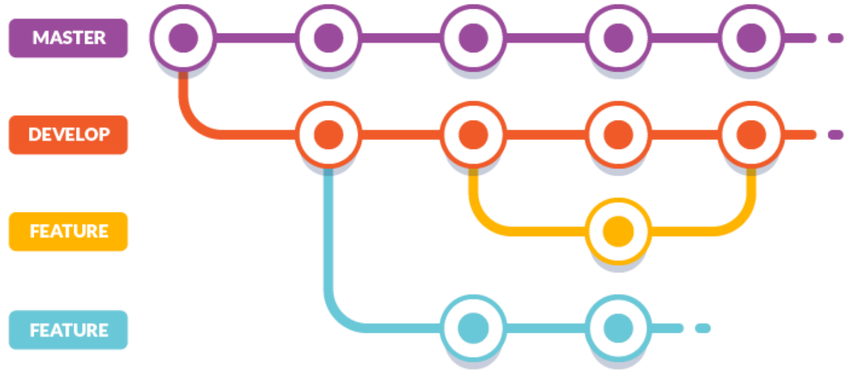
- Unit and Integration tests are run after a commit to a PR
- SonarQube scan runs after the tests
- Passing is essential for a PR to be mergeable

Automation in CD

- Every push to main of the parent repository automatically pulls on deployment and rebuilds the docker compose



Version Control



- Various repositories managed in a central one via github submodules
- All repos have a main and dev branch (GitFlow)
- Releases are gonna be done on the parent repo

PR Workflow

- Features are developed in a specific branch
- After the feature is ready a PR to dev is open that requires review by an individual other than the one who opened the PR (mandatory through a ruleset)

PRs in our Project

146 PRs were made throughout all repositories

Critical bugs managed to pass through due:

- to trusting PRs and not properly reviewing them in time crunches
- Reviewer lacking experience in the tools or frameworks used
- Reviewer working in a completely different part of the project

Questions?

Thank You!

Project Microsite: